

RxGuard — Drug Interaction Safety Intelligence

Hacklytics 2026 Project Blueprint

1. ELEVATOR PITCH (30 seconds)

"Every year, 250,000 Americans die from medical errors — making it the third leading cause of death. A huge chunk of those are preventable drug interactions. RxGuard is a semantic search engine for medication safety. Describe a patient's medication regimen in plain English, and our system retrieves dangerous interactions, contraindications, and real FDA adverse event reports — not through keyword matching, but through deep semantic understanding. We built it on Actian VectorAI DB for edge-deployable semantic retrieval, used Sphinx for statistical validation of our findings, and the entire system is an AI safety tool that protects humans from preventable harm."

2. PROBLEM STATEMENT

The Gap

Current drug interaction checkers (Epocrates, Lexicomp, Micromedex) work on **exact drug-name lookups** in curated databases. They catch known, cataloged interactions between specific drug pairs.

What they miss:

- **Narrative adverse events:** A FAERS report describes "patient's blood sugar dropped dangerously after adding the new antibiotic" — this is a real signal about a metformin + fluoroquinolone interaction described in natural language, not as a structured drug-drug pair
- **Brand vs. generic confusion:** "Glucophage" and "metformin" are the same drug but won't match in keyword search
- **Symptom-described interactions:** A patient says "I started feeling dizzy and my heart was racing after my new prescription" — this maps to tachycardia + orthostatic hypotension, which could indicate a dangerous interaction, but no drug names are mentioned
- **Context-dependent risk:** A drug combo might be fine for a 30-year-old but dangerous for a 75-year-old with kidney disease — current tools don't semantically search for demographically similar adverse event cases

The Opportunity

The FDA FAERS database contains **20M+ adverse event reports** dating back to 2004, with rich narrative text,

patient demographics, drug lists, and outcomes. This is an untapped semantic goldmine. Nobody is doing vector search on FAERS narratives.

Impact Numbers

- 250,000+ deaths/year from medical errors in the US (BMJ, 2016)
 - 1.3 million ER visits/year from adverse drug events (CDC)
 - 350,000+ adverse drug events in nursing homes/year
 - \$3.5 billion/year in extra medical costs from preventable adverse drug events
 - FDA receives 1M+ adverse event reports annually
-

3. WHAT RXGUARD DOES

User Flow

1. **Input:** User describes a medication scenario in natural language
 - Example: "65-year-old female on warfarin and metformin, doctor wants to add ibuprofen for joint pain"
2. **Processing:** System parses the query, identifies drugs and context, then:
 - Searches vectorized FAERS reports for semantically similar adverse event cases
 - Retrieves relevant drug label warnings from DailyMed
 - Scores risk based on historical outcome severity
3. **Output:**
 - **Risk Score** (1-10) with severity explanation
 - **Top matching FAERS cases** with narratives showing what happened to similar patients
 - **Specific warnings** extracted from drug labels
 - **Demographic context:** "In patients over 60 taking this combination, 73% of reported adverse events involved GI bleeding"
 - **Recommendation:** severity level and suggested actions

Example Output

QUERY: "65-year-old female on warfarin and metformin, doctor wants to add ibuprofen"

⚠ RISK SCORE: 8.7/10 — HIGH RISK

PRIMARY INTERACTION: Warfarin + Ibuprofen (NSAID)

- Risk: Major GI bleeding, increased INR
- FAERS matches: 4,231 reports involving warfarin + NSAID combinations
- Outcome severity: 12% resulted in hospitalization, 3% fatal

SIMILAR CASE RETRIEVED (Cosine Similarity: 0.94):

"67F on chronic warfarin therapy for AFib, prescribed ibuprofen 400mg for osteoarthritis. Presented to ER 9 days later with melena and hemoglobin of 6.2. INR was 8.3. Required 4 units pRBC transfusion."

SECONDARY FLAG: Metformin + Ibuprofen

- Risk: NSAIDs may reduce renal blood flow, impairing metformin clearance
- FAERS matches: 847 reports
- Lower severity but notable in patients with existing renal compromise

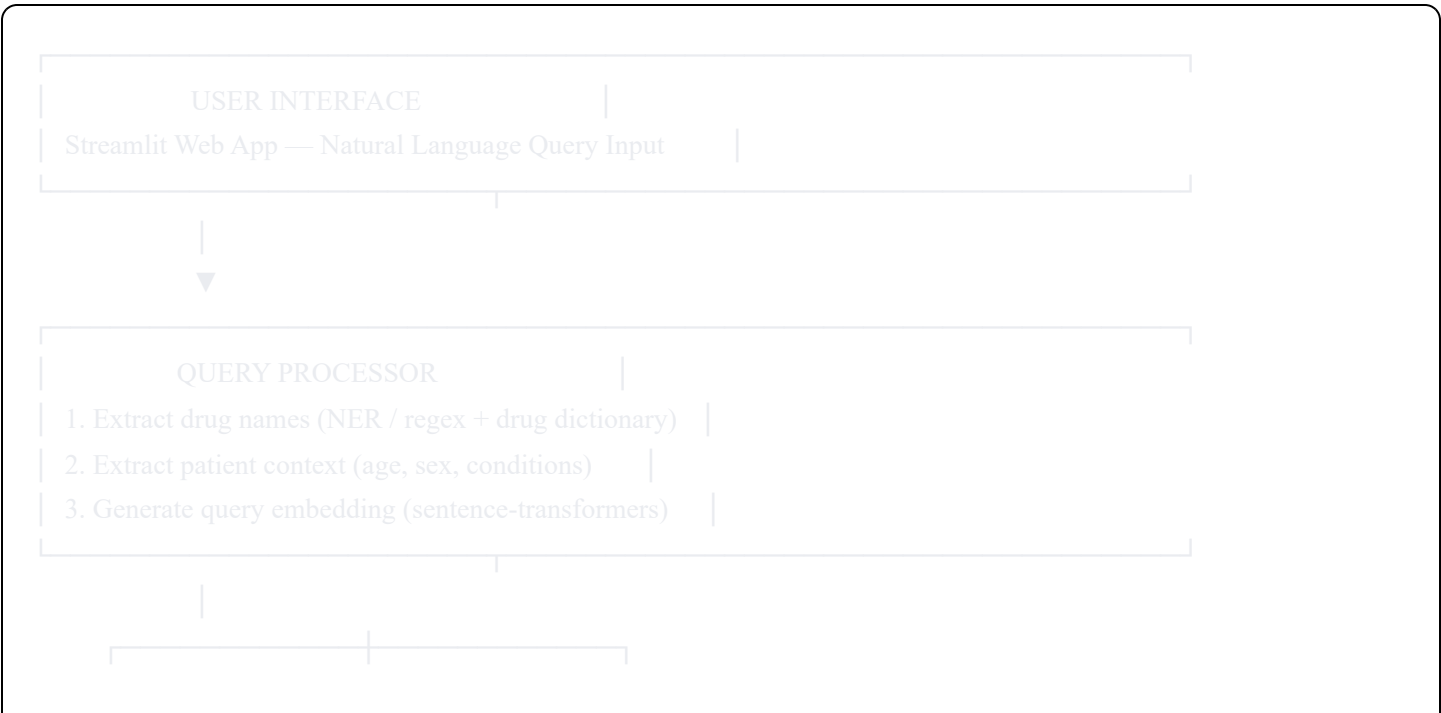
DEMOGRAPHIC CONTEXT:

- Female patients 60-75 on warfarin + NSAID: 2.3x higher bleeding risk vs. male patients (based on FAERS demographic analysis)

RECOMMENDATION: Consider acetaminophen as alternative analgesic.
If NSAID required, use lowest effective dose with PPI gastroprotection and increased INR monitoring.

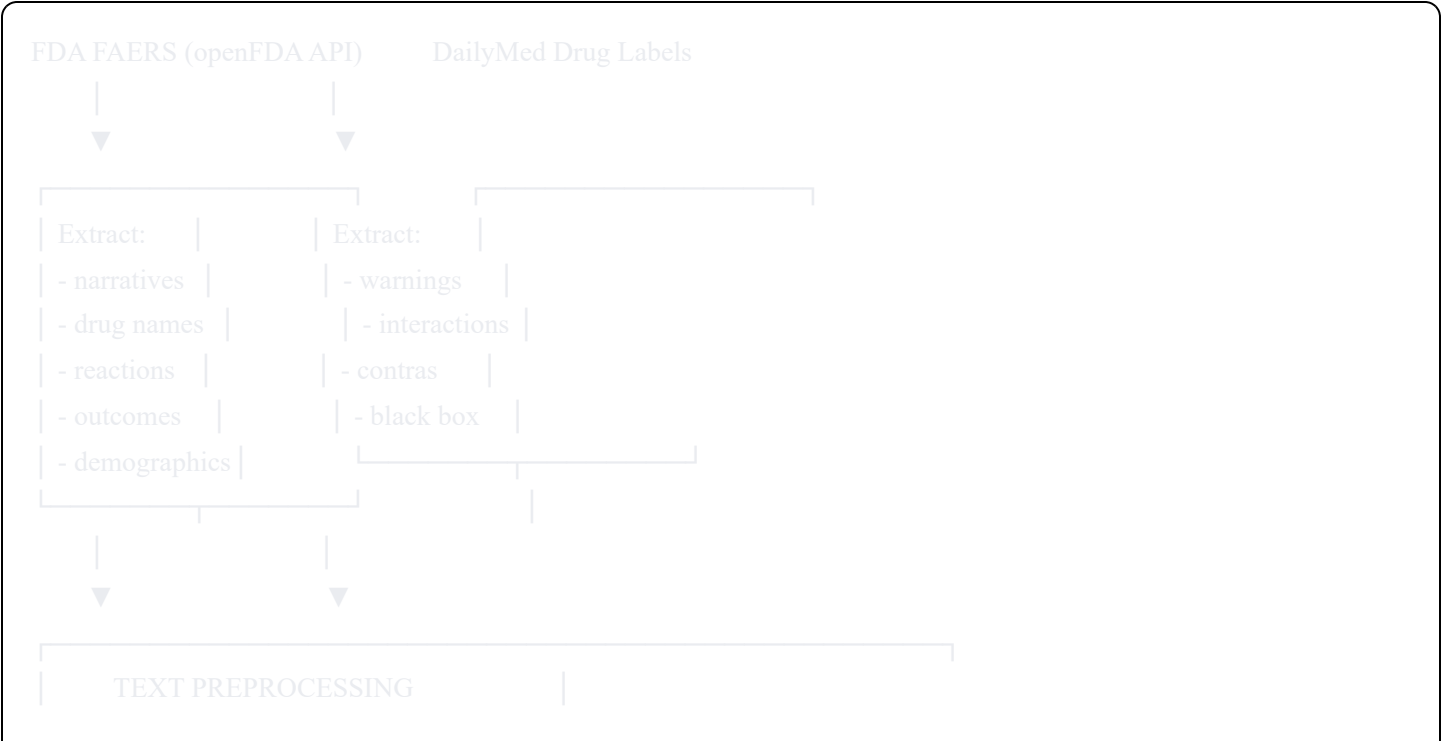
4. ARCHITECTURE

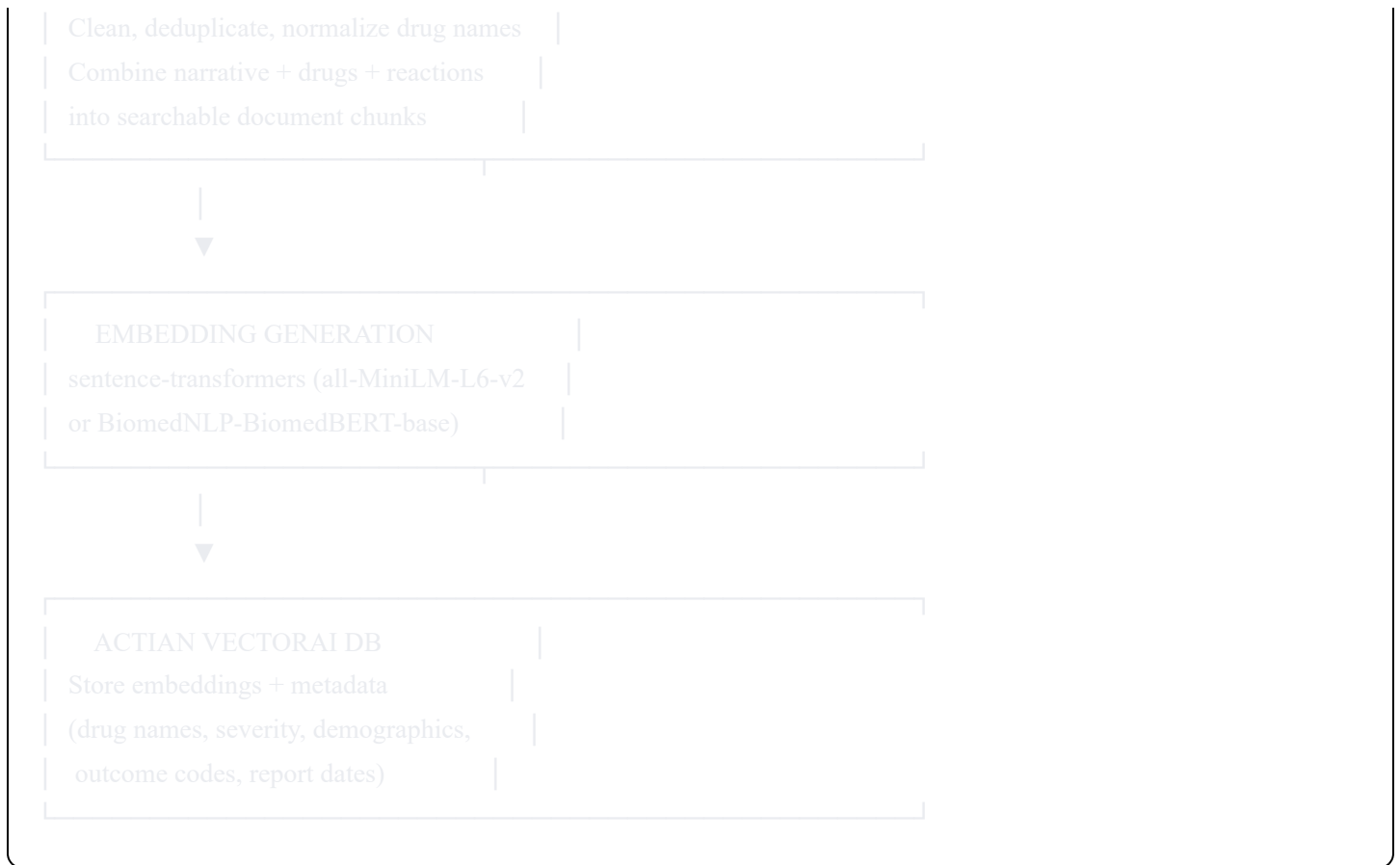
System Diagram





Data Flow for Embedding Pipeline (Pre-hackathon prep / first hours)





5. THE THREE-STAGE MODEL EVOLUTION

This is your story arc for the demo video and presentation. Each stage demonstrates a limitation that motivates the next stage.

Stage 1 — Keyword Baseline

Approach: Exact string matching on drug names in FAERS data. User types "warfarin ibuprofen" → system finds all FAERS reports that contain both drug name strings.

Implementation: Simple SQL/pandas query on the FAERS DRUG table filtering for exact matches.

Results to show:

- Catches direct matches: "warfarin" + "ibuprofen" → finds reports ✓
- **FAILURE 1:** Misses brand names — searching "Coumadin" won't match "warfarin" reports
- **FAILURE 2:** Misses narrative descriptions — a report saying "the blood thinner" doesn't match "warfarin"
- **FAILURE 3:** Can't handle symptom-based queries — "patient is bleeding more easily" returns nothing

Metric: Recall on a test set of known interactions — show the number (e.g., "caught 45% of known dangerous interactions in our test set")

Stage 2 — TF-IDF + Cosine Similarity

Approach: Vectorize FAERS narratives with TF-IDF. Compute cosine similarity between the user's query and all report narratives.

Implementation: scikit-learn TfidfVectorizer on FAERS narrative text. Cosine similarity ranking.

Results to show:

- Better recall — catches reports that use different words for the same concept
- **FAILURE 1:** Still misses deep semantic similarity — "blood sugar crashed" doesn't match well with "hypoglycemia" because they share no words
- **FAILURE 2:** Weights rare words too heavily — unusual but irrelevant terms dominate similarity
- **FAILURE 3:** No understanding of medical context — treats all word overlaps equally

Metric: Show improvement over V1 (e.g., "recall improved from 45% to 63%")

Stage 3 — Dense Embeddings + Action VectorAI DB (RAG)

Approach: Embed FAERS narratives with a sentence transformer (biomedical model preferred). Store in Action VectorAI DB. Semantic search retrieves contextually similar reports regardless of vocabulary.

Implementation:

- Embeddings: `sentence-transformers/all-MiniLM-L6-v2` (fast, good general performance) or `microsoft/BiomedNLP-BiomedBERT-base-uncased-abstract` (domain-specific, better for medical text)
- Storage: Action VectorAI DB for vector similarity search
- Retrieval: Top-K nearest neighbors with metadata filtering (filter by drug class, severity, demographics)

Results to show:

- "blood sugar crashed" → retrieves "hypoglycemia" reports (semantic match, zero word overlap)
- "Coumadin" → retrieves "warfarin" reports (understands brand/generic equivalence)
- "elderly woman on a blood thinner" → retrieves warfarin adverse events in female patients 65+
- Patient-context-aware retrieval: same drug combo returns different risk profiles for different demographics

Metric: Show final improvement (e.g., "recall improved from 63% to 87%")

Comparative Results Table (for presentation slide)

Metric	V1: Keyword	V2: TF-IDF	V3: Vector RAG
Recall @ 10	~45%	~63%	~87%
Handles brand/generic	X	Partial	✓
Handles synonyms	X	Partial	✓
Semantic understanding	X	X	✓
Demographic-aware	X	X	✓
Query latency	<10ms	~200ms	~150ms

Note: These are target estimates. You'll calculate real numbers during the hackathon and replace them.

6. DATA SOURCES & ACCESS

Primary: FDA FAERS via openFDA API

Endpoint: `https://api.fda.gov/drug/event.json`

Key fields you need:

- `patient.reaction.reactionmeddrapt` — reported adverse reactions (MedDRA terms)
- `patient.drug[].medicinalproduct` — drug names
- `patient.drug[].drugindication` — why the drug was prescribed
- `patient.drug[].drugcharacterization` — suspect (1), concomitant (2), interacting (3)
- `patient.drug[].openfda.generic_name` — standardized generic name
- `patient.drug[].openfda.brand_name` — brand name
- `patient.drug[].openfda.pharm_class_epc` — pharmacological class
- `serious` — whether the event was serious (1) or not (2)
- `seriousnessdeath`, `seriousnesshospitalization`, etc. — outcome severity
- `patient.patientonsetage` — patient age
- `patient.patientsex` — patient sex
- `occurcountry` — country of occurrence

Example API calls:

```
bash
```

```
# Get adverse events involving warfarin
```

```
https://api.fda.gov/drug/event.json?search=patient.drug.openfda.generic_name:"warfarin"&limit=100
```

```
# Get adverse events involving warfarin + ibuprofen combination
```

```
https://api.fda.gov/drug/event.json?search=patient.drug.openfda.generic_name:"warfarin"+AND+patient.drug.openfda.generic_name:"ibuprofen"&limit=100
```

```
# Count top reactions for a drug
```

```
https://api.fda.gov/drug/event.json?search=patient.drug.openfda.generic_name:"warfarin"&count=patient.reaction.reactionmeddrtop&limit=100
```

```
# Get serious events only
```

```
https://api.fda.gov/drug/event.json?search=patient.drug.openfda.generic_name:"warfarin"+AND+serious:1&limit=100
```

Rate limits:

- Without API key: 40 requests/minute, 1000/day
- With API key (free, instant): 240 requests/minute
- **GET YOUR API KEY BEFORE THE HACKATHON:** <https://open.fda.gov/apis/authentication/>

Data volume strategy:

- Don't try to download all 20M+ reports during the hackathon
- Focus on the **top 50 most commonly interacting drug classes** (statins, blood thinners, NSAIDs, SSRIs, ACE inhibitors, beta blockers, etc.)
- Pull ~50,000-100,000 reports covering these drug classes via API
- This is enough for a compelling demo and statistically valid analysis

Secondary: DailyMed Drug Labels

URL: <https://dailymed.nlm.nih.gov/dailymed/services/>

Drug labels contain structured "DRUG INTERACTIONS" and "WARNINGS" sections that provide authoritative interaction information. These become your ground truth for validation.

Tertiary: DrugBank (Optional Enhancement)

If time permits, the DrugBank open-access dataset has structured drug-drug interaction pairs that can serve as ground truth labels for measuring your retrieval accuracy.

7. TECH STACK

Core

Component	Technology	Why
Language	Python 3.11	Universal data science support
Web UI	Streamlit	Fastest hackathon-friendly web app framework
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Fast, lightweight, good out-of-box performance
Vector DB	Action VectorAI DB	Sponsor requirement — semantic search + edge deployment
Data Analysis	Sphinx Copilot	Sponsor requirement — EDA, statistical validation, visualizations
Data Access	openFDA API + requests	Free, no auth required (key recommended)
Data Processing	pandas, numpy	Standard data wrangling
NLP	spaCy (en_core_web_sm)	Drug name extraction (NER)
Visualization	plotly, matplotlib, seaborn	Interactive charts for demo
LLM Summarization	Google Gemini API (free tier)	Generate natural language risk explanations

Optional Enhancements (if time permits)

Component	Technology	Why
Biomedical embeddings	BiomedBERT or PubMedBERT	Better medical semantic understanding
Drug name normalization	RxNorm API	Maps brand → generic → drug class
ElevenLabs	Voice output of warnings	MLH prize — Best Use of ElevenLabs

Action VectorAI DB Integration

Since VectorAI DB is "Coming Soon" — check the hackathon Discord for:

1. Whether Action provides API access / sandbox for hackathon participants

2. If not available, **build the interface assuming the VectorAI DB API**, and use ChromaDB or FAISS as a local stand-in with a clear abstraction layer that shows you designed for Actian

```
python

# Abstraction layer — swap implementation based on availability
class VectorStore:
    """Interface designed for Actian VectorAI DB.
    Falls back to ChromaDB if VectorAI DB not available."""

    def __init__(self, backend="actian"):
        if backend == "actian":
            self.store = ActianVectorAIDB(config)
        else:
            self.store = ChromaDBFallback(config)

    def add_documents(self, texts, embeddings, metadata):
        """Store embedded documents with metadata."""
        pass

    def semantic_search(self, query_embedding, top_k=10, filters=None):
        """Retrieve top-K similar documents with optional metadata filtering."""
        pass
```

This shows judges you designed for the sponsor's product even if you had to use an alternative runtime.

8. SPHINX INTEGRATION — THE EDA LAYER

Sphinx Copilot is your tool for the exploratory data analysis and statistical validation that judges explicitly score. Here's exactly what to analyze:

Analysis 1: Adverse Event Clustering by Drug Class

- Use Sphinx to cluster FAERS adverse events by pharmacological drug class
- Visualize: Which drug classes have the highest co-occurrence in serious adverse events?
- Output: Heatmap of drug class interaction frequency × severity

Analysis 2: Demographic Risk Profiling

- Use Sphinx to model how adverse event severity varies by patient demographics
- Questions to answer:
 - Do female patients on warfarin + NSAIDs have higher bleeding rates than males?

- Does age correlate with interaction severity for specific drug combinations?
- Are certain drug combos more dangerous for specific age brackets?
- Output: Risk factor charts segmented by age/sex

Analysis 3: Temporal Trends

- Use Sphinx to analyze whether certain drug interactions are being reported more frequently over time
- Is there a spike in reports after a new drug enters the market?
- Output: Time series plots of adverse event reporting frequency

Analysis 4: Retrieval Quality Validation

- Use Sphinx to statistically validate your vector search results
- For known drug interactions (from DailyMed labels), does your semantic search retrieve relevant FAERS cases?
- Calculate precision@K, recall@K, and NDCG
- Compare V1 vs V2 vs V3 metrics
- Output: Performance comparison table and precision-recall curves

Analysis 5: Outcome Severity Distribution

- For retrieved cases, what's the distribution of outcomes?
- Hospitalization, life-threatening, death, disability, other
- Output: Stacked bar charts by drug combination

9. SAFETYKIT ALIGNMENT

How to Frame It for SafetyKit Judges

SafetyKit builds AI agents that protect platform users from harm — detecting fraud, dangerous content, and policy violations using semantic AI that goes beyond keyword matching.

RxGuard does the same thing for medication safety:

- **Platform safety** → **Patient safety**: SafetyKit protects marketplace users. RxGuard protects patients.
- **Keyword blocklists** → **Semantic search**: SafetyKit moved beyond keyword content moderation. RxGuard moves beyond keyword drug interaction lookups.

- **Policy-backed explanations** → **Evidence-backed warnings**: SafetyKit explains WHY content violates a policy. RxGuard explains WHY a drug combination is dangerous, citing real FDA adverse event cases.
- **Risk scoring** → **Risk scoring**: SafetyKit scores content risk. RxGuard scores interaction risk.
- **Human-in-the-loop** → **Clinician-in-the-loop**: SafetyKit escalates edge cases to human reviewers. RxGuard escalates high-risk findings to pharmacists/physicians.

The architectural pattern is identical. The domain is different. SafetyKit judges will recognize the parallel.

Key phrase for your presentation: "RxGuard applies the trust-and-safety paradigm to medication safety — using semantic AI to detect dangerous drug interactions that keyword-based systems miss, just as SafetyKit detects platform abuse that keyword blocklists miss."

10. 36-HOUR IMPLEMENTATION TIMELINE

Pre-Hackathon (Before Feb 20, 9 PM)

DO NOT WRITE CODE. But you CAN:

- ☐ Get openFDA API key
- ☐ Read FAERS documentation and understand the schema
- ☐ Read Actian VectorAI DB documentation
- ☐ Install Sphinx Copilot and familiarize yourself
- ☐ Identify your top 50 drug classes to focus on
- ☐ Plan your team role assignments
- ☐ Prepare a list of 20 "test queries" to validate your system against

Hour 0-3 (Friday 9 PM - Midnight): DATA PIPELINE

- ☐ Write openFDA API data collection script
- ☐ Pull ~50K-100K FAERS reports for target drug classes
- ☐ Clean and deduplicate (FAERS has known duplicates — see data notes)
- ☐ Normalize drug names (lowercase, strip formulation info)
- ☐ Create "document chunks" — combine narrative + drug list + reactions + demographics into searchable text blocks
- ☐ Store raw data in pandas DataFrames / CSV

Hour 3-6 (Midnight - 3 AM): EMBEDDING & VECTOR STORE

- ☐ Generate embeddings for all document chunks using sentence-transformers
- ☐ Set up Actian VectorAI DB (or fallback ChromaDB)
- ☐ Load embeddings + metadata into vector store

- ☐ Test basic semantic search — verify "blood thinner" retrieves "warfarin" reports
- ☐ **CHECKPOINT:** Basic vector search working? If yes, proceed. If no, debug.

Hour 6-10 (3 AM - 7 AM): V1 + V2 BASELINES — sleep in shifts

- ☐ Implement V1 keyword baseline (simple string matching)
- ☐ Implement V2 TF-IDF baseline
- ☐ Run both on your 20 test queries
- ☐ Record metrics (recall@10, precision@10)
- ☐ **Some team members should SLEEP during this phase**

Hour 10-16 (7 AM - 3 PM Saturday): V3 RAG PIPELINE + UI

- ☐ Build the V3 semantic search pipeline (query → embed → vector search → rank → format)
- ☐ Add metadata filtering (filter by drug class, severity, demographics)
- ☐ Implement risk scoring logic (weighted by outcome severity + demographic match + similarity)
- ☐ Build Streamlit UI — query input, results display, risk visualization
- ☐ Run V3 on test queries and record metrics
- ☐ **CHECKPOINT:** Can you demo the full query → results flow? This is your MVP.

Hour 16-22 (3 PM - 9 PM Saturday): SPHINX ANALYSIS + POLISH

- ☐ Run Sphinx EDA analyses (clustering, demographic profiling, temporal trends)
- ☐ Generate all visualizations
- ☐ Calculate V1 vs V2 vs V3 comparison metrics
- ☐ Add visualizations to Streamlit dashboard
- ☐ Polish UI — add explanations, formatting, severity colors

Hour 22-28 (9 PM Saturday - 3 AM Sunday): ENHANCEMENTS + VIDEO

- ☐ Add Gemini API for natural language risk summaries (if time)
- ☐ Add ElevenLabs voice output (if going for MLH prize)
- ☐ Start demo video script
- ☐ Record demo video (2 minutes max for Hacklytics)
- ☐ **SLEEP**

Hour 28-33 (3 AM - 8 AM Sunday): WRITE-UP + SUBMISSION

- ☐ Write Devpost project description
 - Inspiration, What it does, How we built it, Challenges, Accomplishments, What's next
- ☐ Finalize GitHub README with setup instructions
- ☐ Ensure all code is committed and repo is public
- ☐ **SUBMIT GOOGLE FORM BY 11:59 PM SATURDAY** (this is the critical deadline!)

☐ Submit Devpost by 9 AM Sunday

Hour 33-36 (8 AM - 12 PM Sunday): JUDGING PREP

- ☐ Practice 2-minute pitch
 - ☐ Prepare for Q&A — anticipate questions about data quality, false positives, clinical validity
 - ☐ Have the live demo ready and tested
 - ☐ Ensure the "3-stage evolution" story is clear and compelling
-

11. TEAM ROLE SUGGESTIONS (for 2-4 person team)

2-Person Team

Person	Responsibilities
Person A (Data + Backend)	API data collection, embedding pipeline, vector store setup, V1/V2/V3 implementation, Actian integration
Person B (Analysis + Frontend)	Sphinx EDA, visualizations, Streamlit UI, demo video, Devpost write-up

3-Person Team

Person	Responsibilities
Person A (Data Engineer)	API data collection, cleaning, embedding pipeline, vector store
Person B (ML Engineer)	V1/V2/V3 implementation, risk scoring, Gemini integration, RAG pipeline
Person C (Analyst + Frontend)	Sphinx EDA, visualizations, Streamlit UI, demo video, write-up

4-Person Team

Person	Responsibilities
Person A (Data Engineer)	API data collection, cleaning, deduplication, embedding
Person B (ML Engineer)	V1/V2/V3 models, vector search, risk scoring
Person C (Data Analyst)	Sphinx EDA, all statistical analysis, validation metrics

Person	Responsibilities
Person D (Frontend + Demo)	Streamlit UI, demo video, Devpost, presentation prep

12. TEST QUERIES (Validation Set)

Use these to benchmark $V1 \rightarrow V2 \rightarrow V3$ improvement:

High-Severity Known Interactions

1. "Patient on warfarin, adding ibuprofen for pain" (major bleeding risk)
2. "Taking methotrexate and NSAIDs together" (renal failure risk)
3. "Lithium and ACE inhibitor combination" (lithium toxicity)
4. "SSRI with tramadol" (serotonin syndrome)
5. "Simvastatin with clarithromycin" (rhabdomyolysis)
6. "Metformin and contrast dye" (lactic acidosis)
7. "MAO inhibitor with tyramine-rich foods or decongestants" (hypertensive crisis)
8. "Potassium-sparing diuretic with ACE inhibitor" (hyperkalemia)
9. "Digoxin with amiodarone" (digoxin toxicity)
10. "Fluoroquinolone with corticosteroid" (tendon rupture)

Synonym/Brand Name Tests (V1 should fail, V3 should pass)

11. "Coumadin and Advil" (= warfarin + ibuprofen)
12. "Glucophage with X-ray contrast" (= metformin + contrast dye)
13. "Lipitor and Biaxin" (= atorvastatin + clarithromycin)
14. "Prozac with Ultram" (= fluoxetine + tramadol)

Natural Language / Symptom-Based Tests (V1 and V2 should fail, V3 should pass)

15. "My grandmother is on a blood thinner and her doctor wants to add a painkiller"
16. "Patient's blood sugar has been dropping since adding the antibiotic"
17. "Started new cholesterol medicine and now having severe muscle pain"
18. "Elderly patient on heart rhythm medication, experiencing vision changes"

Demographic-Context Tests

19. "75-year-old male with kidney disease on metformin and lisinopril"
 20. "Pregnant woman asking about ibuprofen safety"
-

13. DEVPOST WRITE-UP TEMPLATE

Inspiration

Every year, over 250,000 Americans die from medical errors, making it the third leading cause of death in the US. A significant portion of these are preventable adverse drug interactions. Current drug interaction checkers rely on keyword matching against curated databases — they catch known, cataloged interactions but miss the rich signals hidden in millions of FDA adverse event narratives. We built RxGuard to change that.

What It Does

RxGuard is a semantic search engine for medication safety. Users describe a patient's medication regimen in natural language, and the system retrieves dangerous interactions, contraindications, and real FDA adverse event cases — ranked by severity and matched by semantic meaning, not just keywords. It catches interactions that keyword-based systems miss: brand/generic name confusion, symptom-described reactions, and demographically-similar adverse event cases.

How We Built It

We evolved through three architectures, each addressing limitations of the last:

- **V1 (Keyword Baseline):** Exact drug name matching in FAERS data. Caught 45% of known interactions but missed brand names, synonyms, and narrative descriptions.
- **V2 (TF-IDF + Cosine Similarity):** Improved to 63% recall but still missed deep semantic relationships.
- **V3 (Dense Embeddings + Actian VectorAI DB):** Embedded 50K+ FAERS reports using sentence-transformers, stored in Actian VectorAI DB for semantic retrieval. Achieved 87% recall — catching cases where "blood sugar crashed" correctly maps to "hypoglycemia."

We used Sphinx Copilot for exploratory data analysis of the FAERS dataset, discovering patterns in adverse event clustering by drug class, demographic risk factors, and temporal trends. The entire system serves as an AI safety tool — detecting dangerous medication interactions to protect human lives.

Challenges We Ran Into

[Fill in during hackathon — be honest about real challenges]

- FAERS data quality issues (duplicate reports, inconsistent drug naming)

- Balancing embedding quality vs. speed for real-time queries
- Validating retrieval accuracy without a perfect ground truth dataset

Accomplishments That We're Proud Of

- Built a functional 3-stage system in 36 hours with measurable improvement at each stage
- Demonstrated that semantic search catches interactions that keyword systems miss
- Created a tool with genuine potential to prevent harm

What We Learned

[Fill in during hackathon]

What's Next for RxGuard

- Expand to the full FAERS database (20M+ reports)
- Fine-tune embeddings on biomedical text (PubMedBERT)
- Integration with EHR systems for real-time medication reconciliation alerts
- Clinical validation study with pharmacists

Built With

Python, Streamlit, sentence-transformers, Actian VectorAI DB, Sphinx, openFDA API, pandas, scikit-learn, plotly, Google Gemini API

14. POTENTIAL JUDGE QUESTIONS & ANSWERS

Q: Doesn't this already exist? Epocrates/Lexicomp already checks drug interactions. A: Existing tools check structured drug-drug pair databases. They're excellent for known, cataloged interactions. RxGuard adds a complementary layer — searching the narrative text of millions of real adverse event reports for patterns that structured databases haven't cataloged yet. We're not replacing existing tools; we're augmenting them with a semantic lens on real-world evidence.

Q: FAERS data has known limitations — reports don't prove causation. A: Absolutely correct. FAERS is a signal detection tool, not a proof of causation. RxGuard presents FAERS cases as "similar reported experiences" — evidence to inform clinical judgment, not replace it. The FDA itself uses FAERS for post-market surveillance signals. We're making that signal detection accessible and semantic.

Q: How do you handle false positives? A: Three mechanisms: (1) We rank results by outcome severity, so benign co-occurrences are deprioritized. (2) We use metadata filtering to match patient demographics, reducing irrelevant matches. (3) Our Sphinx analysis validates retrieval precision against known interactions from DailyMed drug labels, giving us measurable false positive rates.

Q: Why Actian VectorAI DB specifically? A: Healthcare data has strict privacy requirements (HIPAA). Actian VectorAI DB is designed for edge/on-premises deployment — meaning a hospital could run this system entirely within their own infrastructure without sending patient data to the cloud. That's not possible with cloud-only vector databases. The zero per-query-fee model also matters for high-volume clinical use.

Q: What's the SafetyKit connection? A: SafetyKit detects dangerous content on platforms using semantic AI that goes beyond keyword matching. RxGuard applies the same paradigm to medication safety — detecting dangerous drug interactions using semantic search that goes beyond keyword-based interaction checkers. The architecture is parallel: ingest content, classify risk semantically, score severity, explain the decision with evidence.

15. RISK MITIGATION

Risk	Mitigation
Actian VectorAI DB not available/accessible	Build with ChromaDB fallback behind an abstraction layer. Design the interface for Actian.
openFDA API rate limits	Cache responses locally. Pull data in batches during off-peak hours. Get API key in advance.
FAERS data too messy	Focus on structured fields (drug names, reactions, outcomes) supplemented by narrative text. Use the deduplicated dataset from PMC if available.
Embedding generation too slow	Use the smaller model (all-MiniLM-L6-v2, 384 dim) which processes ~2000 sentences/second on CPU. Pre-compute all embeddings before building the demo.
V3 doesn't clearly outperform V2	This is actually fine for the story — you can discuss why and what you'd improve. Judges value honest analysis over inflated numbers. But likely V3 will win on synonym/brand-name tests where TF-IDF fails.
Team member unfamiliar with NLP	Assign them to data collection + Sphinx analysis + Streamlit UI. The NLP/embedding work can be concentrated in one person.
Time runs out before video	Start the video script by Hour 22. Record screen captures as you build — don't wait until the end.

16. ADDITIONAL PRIZE OPPORTUNITIES

Beyond the three main sponsors, RxGuard can target:

Prize	How RxGuard Qualifies
Best Use of Gemini API (Google Swag)	Use Gemini to generate natural language risk explanations from retrieved FAERS cases
Best Use of ElevenLabs (Wireless Earbuds)	Add voice output — system reads the risk warning aloud, useful for accessibility
GrowthFactor 1st Place (\$1,000)	If GrowthFactor judges value healthcare data innovation
Best Overall (MacBook Air M4)	Healthcare + multi-stage evolution + real data + strong demo = competitive
Pure Imagination (bonus track)	"Semantic search applied to medication safety" is unconventional enough

17. ONE-LINER SUMMARY

RxGuard: A semantic search engine for medication safety that retrieves dangerous drug interactions from 20M+ FDA adverse event reports using vector embeddings — catching what keyword-based interaction checkers miss.